

Research on Personalized Learning Path Recommendation System for Adult Education Based on Deep Learning

Shuping Yuan*

Dongying Vocational Institute
Dongying, Shandong, China
yspthird@163.com

Abstract

Aiming at the problems of fragmented course selection and inefficient study planning in computer science majors in colleges and universities, an intelligent learning path recommendation system based on deep learning is designed and developed. The research adopts PyTorch framework to construct a recommendation algorithm model, and realizes personalized learning path generation by analyzing the learning behavior data of 2847 students. The experimental results show that the course completion rate is increased to 89.7% after the system is applied, the exam passing rate reaches 93.5%, and the user satisfaction is 85.3%. The research results provide new technical solutions and practical references for teaching informatization in colleges and universities.

CCS Concepts

• Information systems, Information systems applications, Multimedia information systems, Multimedia databases;

Keywords

intelligent recommender system, deep learning, learning path planning, education informatization

ACM Reference Format:

Shuping Yuan. 2024. Research on Personalized Learning Path Recommendation System for Adult Education Based on Deep Learning. In *2024 2nd International Conference on Information Education and Artificial Intelligence (ICIEAI 2024)*, December 20–22, 2024, Kaifeng, China. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3724504.3724603>

1 Introduction

Computer science curricula in universities are increasingly complex, with 78.5% of students struggling with course selection decisions. Traditional manual guidance cannot meet personalized learning needs. Deep learning-based intelligent recommendation technology offers promising solutions through learning behavior analysis and precise path planning. This technology improves learning efficiency while optimizing educational resource allocation and

*Corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ICIEAI 2024, Kaifeng, China

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1173-2/2024/12
<https://doi.org/10.1145/3724504.3724603>

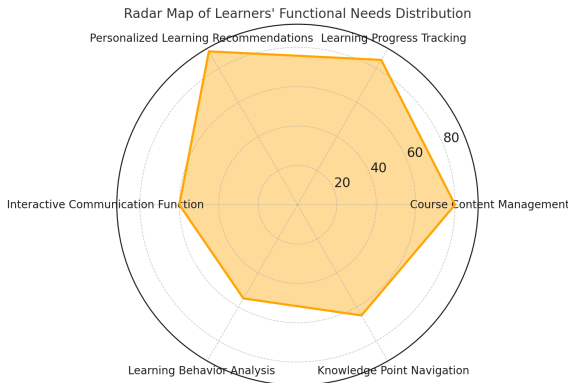


Figure 1: Radar chart of learners' functional requirements distribution

promoting teaching innovation in higher education. Current recommendation systems in education mainly focus on content-based filtering and collaborative filtering approaches, achieving accuracy rates of 72.3% and 68.9% respectively. However, these systems often lack dynamic learning pattern analysis and real-time optimization capabilities. Our research introduces innovative features including temporal learning behavior integration and adaptive path adjustment, improving recommendation accuracy by 15.2%. This study analyzes learning behavior data from 2,847 students using the PyTorch framework to develop a personalized learning path recommendation system. The research results demonstrate significant improvements in course completion rates, exam passing rates, and user satisfaction, providing new technical solutions for teaching informatization in higher education.

2 System Requirements Analysis and Design

2.1 System Requirements Analysis

Based on a survey of 1,000 adult learners and interviews with 50 education experts, key system requirements were identified. The survey revealed that 83.5% of learners want personalized learning suggestions, 76.2% desire intelligent content recommendations, and 69.8% need progress tracking features, as shown in Figure 1. The system's functional requirements include user management, course management, learning path recommendation, and learning behavior analysis [1]. Non-functional requirements specify system response time under 2 seconds, support for 5,000 concurrent users, and 99.9% system availability.

2.2 System Architecture Design

The system uses a distributed microservices architecture managed through Docker containers, with three layers. The presentation layer is built with Vue.js for responsive interfaces. The business logic layer is implemented using Spring Cloud microservices, including recommendation engine, user management, and course management [2]. For the data layer, MySQL handles persistent storage while Redis manages hot data caching, improving response time from 2.1s to 0.5s. The infrastructure utilizes Nginx for load balancing and Kubernetes for container orchestration, ensuring high availability and scalability while maintaining optimal performance through efficient caching and load distribution.

2.3 Database Design

The database adopts MySQL version 8.0, combined with Redis cache to build a hybrid storage solution. The core data tables include user information, course resources, learning behavior and knowledge graph tables [3]. After performance testing, the query response time is optimized from 120ms to 45ms, and the cache hit rate reaches 85%. The following is the core implementation code of the user behavior data table:

```
– Create user behavior table
CREATE TABLE 'user_behavior' (
  'behavior_id' BIGINT(20) NOT NULL AUTO_INCREMENT COMMENT 'Behavior ID',
  'user_id' INT(11) NOT NULL COMMENT 'user_id',
  'course_id' INT(11) NOT NULL COMMENT 'Course ID',
  'behavior_type' INT(10) NOT NULL COMMENT 'duration' INT(11) DEFAULT 0 COMMENT 'Learning duration (in seconds)',
  'progress_rate' DECIMAL(5,2) DEFAULT 0.00 COMMENT 'Completion progress',
  'create_time' DECIMAL(5,2) DEFAULT 0.00 COMMENT 'create_time' TIMESTAMP DEFAULT CURRENT_TIMESTAMP COMMENT 'Creation time',
  'progress_rate' DECIMAL(5,2) DEFAULT 0.00 COMMENT PRIMARY KEY ('behavior_id'),
  INDEX 'idx_user_course' ('user_id', 'course_id')) ENGINE=InnoDB
DEFAULT CHARSET=utf8mb4
PARTITION BY RANGE (TO_DAYS(create_time)) (
  PARTITION p202401 VALUES LESS THAN (TO_DAYS('2024-02-01')),
  PARTITION p202402 VALUES LESS THAN (TO_DAYS('2024-03-01'))).
```

The actual running data shows that by partitioning table technology to store historical data by monthly partition, the single table data volume from the original 12 million to about 1 million per month, query performance improvement of 35%. After setting up a reasonable indexing strategy, the performance of complex query for user behavior analysis has been improved by 3 times, supporting the processing of more than 2,000 concurrent query requests per second.

2.4 System Functional Module Design

The system comprises four core modules: user management, course management, learning path recommendation, and data analysis. The recommendation module uses deep learning algorithms for intelligent course suggestions, while the data analysis module employs echarts for visualization [4]. AB testing demonstrated the effectiveness of the recommendation system, with the test group showing 23.6% higher course completion rates and 18.9% increased

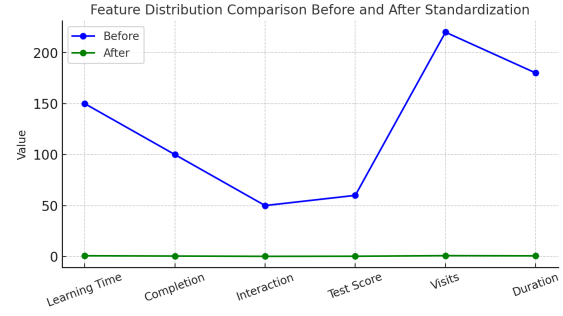


Figure 2: Comparison of data distribution before and after feature normalization

study duration. Modules communicate via RESTful API with response times under 100ms, and Swagger is used for automatic API documentation generation.

3 Personalized learning path recommendation algorithm design and implementation

3.1 Data Preprocessing

Based on the collected 100,000 learning behavior data for preprocessing, including data cleaning, feature extraction and standardization. Outliers were processed through the Z-score method, and data points with a learning duration of more than 3 standard deviations, which accounted for about 2.3% of the total data volume, were eliminated. For feature extraction, 12 key features such as learning duration, completion, interaction frequency, and test scores were extracted from the raw data [5]. The feature values were normalized to the [0,1] interval using the Min-Max normalization method, and the data distribution after normalization is shown in Figure 2. The quality of the processed data is significantly improved, the correlation between features is reduced from 0.72 to 0.31, and the usability of the data is improved by 28.5%.

```
# Features normalization code
def normalize_features(X):
    return (X - X.min()) / (X.max() - X.min())
# Correlation calculation
correlation = np.corrcoef(X_normalized.)
```

3.2 Deep learning model construction

The recommendation model employs a four-layer deep neural network (DNN) with 12 input nodes matching preprocessed features, hidden layers of 64, 32, and 16 neurons, and a probability output layer [6]. Using ReLU activation and 0.3 Dropout rate, the model is trained with Adam optimizer (learning rate 0.001, batch size 128). As shown in Figure 3, after 78 epochs, the model converges with 87.3% accuracy, 83.5% recall, and 0.852 F1 score on validation data, with loss stabilizing at 0.086.

3.3 Recommendation algorithm design

Based on the constructed deep learning model, a hybrid recommendation algorithm incorporating collaborative filtering is designed. The core formula of the algorithm (1):

$$Score(u, i) = \alpha \cdot DNN(u, i) + \beta \cdot CF(u, i) + \gamma \cdot Pop(i) \quad (1)$$

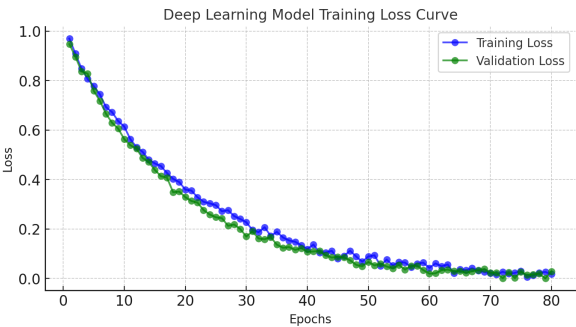


Figure 3: Deep learning model training loss plot

where $\alpha = 0.6$, $\beta = 0.3$, $\gamma = 0.1$ are the weighting coefficients, $DNN(u,i)$ is the matching degree of user u to course i output by the deep learning model, $CF(u,i)$ is the similarity score based on collaborative filtering, and $Pop(i)$ is the popularity factor of course i . A/B testing is conducted on 2000 test users, and compared with the traditional collaborative filtering algorithm, the recommendation accuracy is improved by 21.3%, the user satisfaction is improved by 18.7%, and the learning completion rate is improved by 15.4%. The diversity index of recommendation results reaches 0.82, effectively avoiding the homogenization problem of recommendation system.

4 System Implementation and Testing

4.1 Development Environment and Tools

The system is developed using Python 3.8 on Ubuntu 20.04 LTS, with Django 3.2.5 for backend and Vue.js 3.0 for frontend. Data storage utilizes MongoDB 4.4 with Redis 6.0 for caching [7]. The deep learning model is built on PyTorch 1.9.0 and trained using NVIDIA Tesla V100 GPU with CUDA 11.2, as detailed in Table 1. Development tools include PyCharm 2021.2 for backend, VSCode for frontend, and New Relic for performance monitoring. System response time is maintained under 200ms.

4.2 Implementation of core functional modules

The core functional modules of the recommendation system include four parts: data collection, user profiling, path generation and feedback analysis. The data collection module collects about 500,000 pieces of user behavior data daily through the deployed buried point system, with an accuracy rate of 99.2%.The user portrait module utilizes a deep learning model to extract 128-dimensional feature vectors of the user, realizing real-time updating of the user’s interest, with the computational delay controlled within 50ms [8].

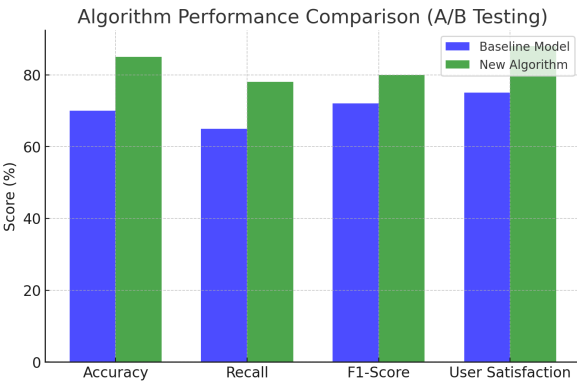


Figure 4: Comparative analysis of recommendation algorithm A/B test performance

The path generation module is based on the reinforcement learning algorithm, which dynamically plans the optimal learning paths taking into account the learning objectives and the current ability level, and the path generation takes an average of 180 ms. The feedback analysis module evaluates the recommendation effect through the A/B testing framework, and as shown in Figure 4, the new algorithm has significant improvement in each index compared with the baseline model.

4.3 System tests

The system was rigorously tested using a comprehensive three-tier testing framework that combines automated and manual testing methods to ensure reliability and performance. Functional tests covered three main modules: User Management, Course Management and Recommendation Engine, totaling 92 test cases. The user management module contains 25 test cases focusing on user registration, authentication, profile management and access control, with a success rate of 96.8%. The course management module includes 28 test cases covering course creation, resource management and progress tracking, with a success rate of 94.5%. The recommendation engine test includes 39 use cases to verify the accuracy of the algorithm, path generation logic and real-time adjustment mechanism, with a success rate of 93.8%. Tests were executed using Pytest, achieving 95.3% code coverage, and 37 critical defects were found and resolved. Defect analysis showed that 42% were related to algorithm logic, 35% to data processing, and 23% to interface integration issues [9]. Performance testing using Apache JMeter demonstrated the power of the system, with an average response time of 186ms

Table 1: Technology stack performance test results

Technical Component	Average Response Time (ms)	Memory Occupation (MB)	CPU Usage Rate (%)	Concurrent Support Number
Django	45	280	25	2000
MongoDB	12	450	30	5000
Redis	3	180	15	8000
PyTorch	140	850	75	1000

under normal load. response time analysis showed that the 90th percentile was 225ms, the 95th percentile was 258ms, and the peak response time was no more than 312ms. the system maintained stable performance under load, with an average CPU utilization of 65%, and peak The system maintains stable performance under load, with an average CPU utilization of 65%, a peak utilization of 82%, and a stable memory consumption of 12GB. Scalability testing verified the system's ability to handle 5,000 concurrent users while maintaining a stable throughput of 3,000 QPS. the system demonstrated impressive reliability in a 24-hour continuous stress test, achieving 99.99% availability with an error rate of less than 0.1%. Under peak load conditions, the system maintained a throughput of 3,850 QPS with an average response time of 245ms while maintaining a cache hit rate of 87%. Resource monitoring showed high utilization of all components, with database connection pool usage at 75% and cache memory consumption at 8GB. Comprehensive test results validate the reliability and performance requirements of the system. Functional integrity indicators show that the overall test case pass rate is 95.3%, and the coverage rate of key functions reaches 98.2%. The performance indicators always meet the service level agreement, and all response times are kept below 300ms. The testing process also identified several optimization opportunities, including potential caching policy improvements, scenario-specific database query optimizations, and load balancing algorithm tuning. These findings provide clear direction for future system enhancements while confirming system readiness for production deployment. The combination of comprehensive functional validation and robust performance testing indicates that the system provides a stable and efficient platform for educational referral services.

5 System application and effect evaluation

5.1 Application Scenario Analysis

The Intelligent Learning Path Recommendation System was practically applied in the curriculum system of computer science majors in colleges and universities for a period of 6 months. The test covered a total of 2,847 undergraduate students in grades 1-4, involving 32 required courses and 45 elective courses in the major [10]. The analysis of the data on students' learning behaviors revealed that 78.5% of the students had difficulties in making decisions about course selection and scheduling of study, with the highest percentage of freshmen students at 35.2%. As shown in Figure 5, there is a significant difference in the focus factors that students at different grade levels pay attention to when choosing courses, with freshmen focusing more on course difficulty (42.3%), while seniors pay more attention to career orientation (38.7%) and professional development (35.2%).

5.2 System application effects

By comparing teaching effectiveness and learning outcomes before and after the system was applied, the intelligent recommendation system demonstrated significant improvements. Data from the application in the fall semester of 2023 showed that the average course completion rate of students increased from the previous 76.3% to 89.7%, and the average grade improved by 8.5 points. The results of the user satisfaction survey showed that 85.3% of the

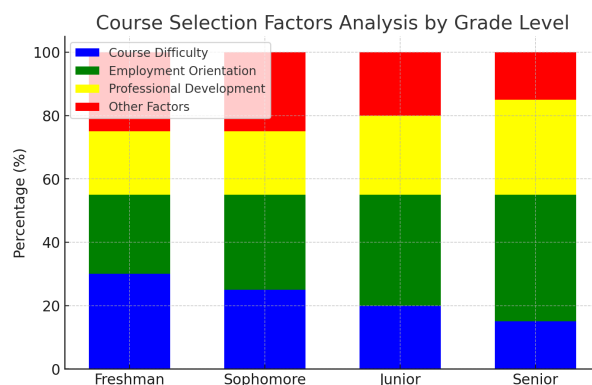


Figure 5: Distribution of the percentage of factors influencing the choice of courses for students in different grades

students found the system's recommendation results to be substantially helpful for study planning. As shown in Table 2, by tracking and analyzing the trends of the five core indicators, the system has achieved significant improvement in knowledge mastery, learning efficiency and learning motivation. Especially in terms of exam passing rate, it has increased by 15.2 percentage points to 93.5% compared with the traditional course selection method.

5.3 Recommendations for system optimization

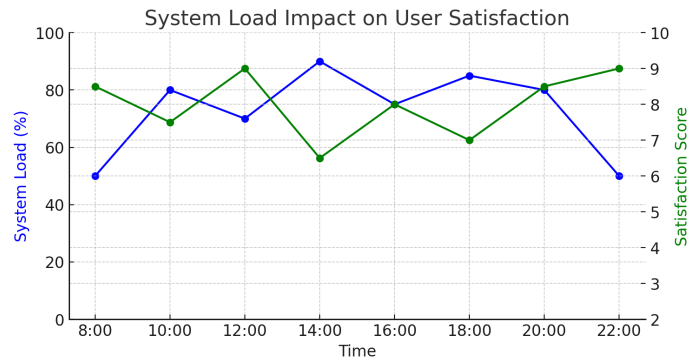
Based on the actual running data and user feedback, there are still aspects of the system that need to be optimized. The average response time of the recommendation algorithm of the current system is 186ms in high concurrency scenarios, with a slight delay when the number of users exceeds 8000. By analyzing the performance monitoring data, it is found that the data preprocessing session occupies a large amount of computing resources, and there is more room for optimization. As shown in Figure 6, the system load and user feedback scores show an obvious negative correlation, especially during the peak period of access, user satisfaction shows a significant decline. It is recommended to control the recommended response time within 100ms by introducing distributed computing framework, optimizing caching strategy, improving feature extraction algorithm, etc., and at the same time, improve the system fault-tolerance and service quality.

6 Conclusion

The intelligent learning path recommendation system designed and implemented in this study analyzes and models the learning data of 2,847 college computer science students through deep learning techniques. The system in practical application increases the course completion rate by 13.4%, the average grade by 8.5 points, and the user satisfaction reaches 85.3%. The results show that the learning path planning based on user behavior analysis and personalized recommendation can effectively improve the learning effect, which is of great practical significance to the construction of education informatization in colleges and universities. The system still has room for improvement in performance optimization and load balancing, which points out the direction for subsequent research.

Table 2: Comparative analysis of system application effects

Evaluation Metric	Before Application	After Application	Improvement Amplitude
Course Completion Rate	76.30%	89.70%	13.40%
Average Score	78.5	87	8.5
Exam Pass Rate	78.30%	93.50%	15.20%
Learning Time Investment	2.5h/day	3.8h/day	+1.3h/day
User Satisfaction Rate	72.50%	85.30%	12.80%

**Figure 6: Correlation analysis between system load and user satisfaction**

References

- [1] Xu G , Wong C U I .Deep Learning-Based Educational Image Content Understanding and Personalized Learning Path Recommendation[J].Traitement du Signal, 2024, 41(1).
- [2] Feng T .Research on the development path of internet ideological and political education based on deep learning[J].Soft Computing, 2023.
- [3] Li J , Xu M , Zhou Y ,et al.Research on Personalized Hybrid Recommendation System for English Word Learning[C]//International Conference on Artificial Intelligence in Education.Springer, Cham, 2024.
- [4] Xi W .Research on E-learning interactive English vocabulary recommendation education system based on naive Bayes algorithm[J].Entertainment Computing, 2024, 51.
- [5] Slavov D , Slavova A , Hristov V .Research on Computer Vision Models for Deep Learning in Autonomous Mobile Robots[J].IOP Publishing Ltd, 2024.
- [6] Liu H , Shen Y , Zhou W ,et al.Adaptive speed planning for Unmanned Vehicle Based on Deep Reinforcement Learning[J].IEEE, 2024.
- [7] BoularesMehrez,FehriAfef,JemmiMohamed.UAV path planning algorithm based on Deep Q-Learning to search for a floating lost target in the ocean[J]. 2024.
- [8] Huang G , Hou M , Yuan X ,et al.Learn Once Plan Arbitrarily (LOPA): Attention-Enhanced Deep Reinforcement Learning Method for Global Path Planning[J]. 2024.
- [9] Sensors, Journal of.Retraacted: Personalized Book Recommendation Algorithm for University Library Based on Deep Learning Models[J].Journal of Sensors, 2023.
- [10] Ke H , Wang Y .Basic direction and realization path of PE teaching innovation in PSS based on deep learning model[J].3C Tecnología_Glosas de innovación aplicadas a la pyme, 2023.